Knowledge Organiser — 1.4 Data Types, Data Structures & Boolean Algebra

OCR A Level Computer Science (H446) · Component 01 · 1.4.1 Data types · 1.4.2 Data structures · 1.4.3 Boolean algebra

1.4.1 Data Types

Primitive types

Type	Holds	Example
Integer	Whole number (no fraction)	`42`, `-7`
Real / Float	Number with fractional part	`3.14`
Char	Single character	`'A'`
String	Sequence of characters	`"Hi"`
Boolean	True / False	`True`

Binary place values (8-bit unsigned, range 0–255)

128	64	32	16	8	4	2	1

Hex = base 16 (0–9, A=10…F=15). **1 hex digit = 4 bits (nibble)**; 2 hex = 1 byte.

Conversion methods

- **Denary → binary:** subtract largest place value that fits, repeat (or ÷2, read remainders bottom-up).
- **Binary → denary:** add place values where bit = 1.
- **Binary → hex:** split into nibbles **from the right** (pad left with 0s), convert each.
- **Hex → binary:** expand each digit to 4 bits. **Hex → denary:** $(\text{digit}_1 \times 16) + \text{digit}_0$.
- *Why hex?* Compact, lossless, error-free shorthand for binary (addresses, colours, machine code).

Two's complement (signed) — 8-bit range −128 to +127

−128	64	32	16	8	4	2	1

- MSB has a **negative** place value; MSB = 1 ⇒ negative.
- **Negate (find −x): invert ALL bits, then ADD 1.** e.g. +5 `00000101` → invert `11111010` → +1 → `11111011` = −5.

Binary addition rules

`0+0=0` · `1+0=1` · `1+1=10` (write 0 carry 1) · `1+1+1=11` (write 1 carry 1) - **Overflow:** carry beyond available bits / two same-sign numbers give a wrong-sign result.

Binary subtraction

- **A − B = A + (−B)** — add the two's complement of B; **discard final carry** out of MSB.

Binary shifts

Shift	Effect
Logical left × n	fill 0s from right → **× 2^n**
Logical right × n	fill 0s from left → **÷ 2^n** (integer; lost bits)
Arithmetic right × n	**copies sign bit** in from left → **÷ 2^n for signed** numbers
- Bits shifted off the end are **lost**; left shift can overflow.	

Bitwise ops

AND = mask/keep bits · **OR** = set bits to 1 · **XOR** = toggle/differ · **NOT** = invert.

Floating point — value = mantissa × 2^exponent

- **Mantissa** = significant figures → **precision**. **Exponent** = position of point → **range**.
- Both stored in **two's complement** (signed). Point assumed after first (sign) bit of mantissa.
- **Normalisation rule:** positive starts `0.1…` ; negative starts `1.0…` (removes wasted leading bits → max precision).
- Moving point **right ⇒ exponent decreases**; moving **left ⇒ exponent increases** (value unchanged).
- **Trade-off (fixed total bits):** more mantissa = more precision, less range; more exponent = more range, less precision.

Character sets

- **ASCII:** 7-bit = 128 chars (`'A'`=65, `'a'`=97, `'0'`=48); extended 8-bit = 256.
- **Unicode:** more bits (UTF-8 = 1–4 bytes) → ~1.1M code points, all languages + emoji. **ASCII is a subset** (first 128 codes) ⇒ backward-compatible.

1.4.2 Data Structures

Static vs dynamic

	Static	Dynamic
Size	Fixed at declaration	Grows/shrinks at run time
Memory	Contiguous block up front	Allocated from **heap** as needed
Trade-off	Fast access, may waste/run out	Flexible, pointer overhead
Example	**Array**	**Linked list**

Structure	Key facts & operations	Static/Dynamic
Array (1D/2D/3D)	Fixed size, **same type**, indexed (0-based), O(1) access. 2D `grid[r][c]`, 3D `cube[l][r][c]`	Static
Record	Named fields of **different** types; `student.name`	Static
List	Ordered, **mutable**, mixed types, resizable; append/insert/remove/index	Dynamic
Tuple	Ordered, **immutable** (cannot change); e.g. `(x,y)`	Static
Linked list	**Nodes** = data + pointer to next; head pointer, last = null. Easy insert/delete (repoint); **no random access**, must traverse	Dynamic
Stack (LIFO)	push/pop/peek at **top** via stack pointer. Overflow/underflow. Uses: call stack, undo, backtracking	Either
Queue (FIFO)	enqueue at **rear**, dequeue at **front**. Circular: `rear=(rear+1) MOD size` reuses space. Priority: highest priority first. Uses: spooling, buffers, scheduling	Either
Graph	Vertices + edges; directed/undirected/weighted. Adjacency **matrix** (dense, O(1) lookup) vs **list** (sparse, memory-efficient)	Either
Tree	Connected, acyclic; root, parent/child, leaf. Binary tree ≤ 2 children	Either
BST	Binary tree where **left subtree < node < right subtree**; O(log n) search when balanced. Insert: go left if smaller, right if larger	Either
Hash table	Key→index via **hash function** (e.g. `key MOD size`), ~O(1). **Collision** = 2 keys same index → linear probing or chaining. Load factor ↑ = slower	Either

Tree traversals: Pre-order (root,L,R) prefix/copy · **In-order (L,root,R) = BST ascending** · Post-order (L,R,root) postfix/delete.

1.4.3 Boolean Algebra

Logic gates truth table

A	B	AND	OR	XOR	NAND	NOR
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

NOT ¬A: inverts. AND `A·B` · OR `A+B` · XOR `A⊕B` · NAND `¬(A·B)` · NOR `¬(A+B)`. NAND & NOR are **universal**.

Boolean identities

Law	Form
Identity	A·1 = A · A+0 = A
Null	A·0 = 0 · A+1 = 1
Idempotent	A·A = A · A+A = A
Complement	A·Ā = 0 · A+Ā = 1
Double negation	¬(¬A) = A
Commutative	A·B = B·A · A+B = B+A
Associative	A·(B·C) = (A·B)·C · A+(B+C) = (A+B)+C
Distributive	A·(B+C) = A·B + A·C · A+(B·C) = (A+B)·(A+C)
Absorption	A + A·B = A · A·(A+B) = A
De Morgan's	**¬(A·B) = ¬A + ¬B · ¬(A+B) = ¬A · ¬B**

De Morgan's: "break the bar, change the sign" — negate **each** variable AND swap AND↔OR. **K-maps:** group adjacent 1s in powers of 2 (1,2,4,8…), as large as possible, may wrap/overlap → minimal sum-of-products.

Adders

Half adder (2 bits, no carry-in): - **Sum = A ⊕ B** · **Carry = A · B**

Full adder (A, B, Cin) = two half adders + OR; chain → ripple-carry adder: - **Sum = A ⊕ B ⊕ Cin** - **Cout = (A·B) + (Cin·(A⊕B))**

D-type flip-flop

- **Edge-triggered 1-bit memory** (sequential logic). On active **clock edge**, stores input **D** and presents it at **Q** until next edge; synchronises data to the clock.
- Uses: registers, counters, memory cells. (Combinational = output from inputs only; sequential = inputs + stored state.)

Must-know definitions

Term	Definition
Two's complement	Signed-integer rep where MSB is a negative place value; negate = invert all bits + 1
Overflow	Result too large for the available bits
Mantissa	Significant-figures part of a float; sets **precision**
Exponent	Power-of-two part of a float; sets **range**
Normalisation	Float adjusted so mantissa begins with first significant bit (0.1… +ve / 1.0… −ve) for max precision
Logical shift	Fills with 0s; ×/÷ unsigned by powers of 2
Arithmetic shift	Right shift preserving sign bit; ÷ signed by powers of 2
Character set	Mapping of characters to unique binary codes (ASCII, Unicode)
Array	Ordered, fixed-size, same-type collection accessed by index
Record	Collection of named fields, possibly different types
Tuple	Ordered, immutable sequence
Linked list	Dynamic list of nodes, each = data + pointer to next
Stack / Queue	LIFO (push/pop at top) / FIFO (enqueue rear, dequeue front)
BST	Binary tree where left subtree < node < right subtree
Hash table	Keys → indices via hash function for fast lookup; collision = 2 keys same index
Half / Full adder	Adds 2 bits → Sum (XOR) + Carry (AND) / adds 3 bits (incl. Cin) → Sum + Cout
D-type flip-flop	Edge-triggered 1-bit memory storing D on a clock edge

Top exam reminders (discriminators)

- **Two's complement negation:** invert **all** bits **then +1** — don't forget the +1; in subtraction **discard the final carry**.
- **Shifts:** always state **direction AND effect** (× or ÷ 2^n); bits shifted off are **lost**; arithmetic right keeps the **sign bit**.
- **Normalisation:** +ve starts `0.1`, −ve starts `1.0`; point **right ⇒ exponent down**, left ⇒ up. **More mantissa = precision, more exponent = range** (don't swap).
- **De Morgan's:** change the operator **and** negate **each** variable (not just one); tidy with double negation.
- **Structures:** BST ≠ any binary tree (needs ordering property); name LIFO vs FIFO ends correctly; half adder has **no carry-in**, full adder adds three bits.